

Machine Learning in Epidemiology

An Introduction with Case Study of Predicting Extreme Centenarianism

Dr. Dor Atias, MD-MPH

2025-10-07

Learning Objectives

By the end of this lecture, you will be able to:

- **Distinguish** between machine learning and traditional epidemiological approaches
- **Understand** core ML concepts: the ML workflow, models, overfitting, cross-validation
- **Implement** ML workflows using R and tidymodels
- **Evaluate** model performance using appropriate metrics
- **Apply** explainable AI tools to interpret complex models
- **Recognize** when ML is appropriate vs. traditional methods
- **Create** reproducible ML pipelines for public health applications

This lecture bridges traditional epidemiological training with modern ML methods. We'll use the Centenarianism prediction study throughout as our practical example.

Why Machine Learning in Public Health?

What is Machine Learning?

“The field of study that gives computers the ability to learn without being explicitly programmed”

— **Arthur Samuel (1959)**

Start with the classic definition from Arthur Samuel, one of the pioneers of machine learning. This emphasizes that ML is about pattern recognition and learning from data, not just following pre-programmed rules.

Prediction vs. Causation: Different Tools for Different Questions

Traditional Epidemiological Methods

- **Causal inference** focus: understanding mechanisms
- Clear **parametric assumptions**
- Need for **Pre-specified interactions**
- **Linear** relationships
- Relatively parsimonious models
- **Interpretable** coefficients

*“Does smoking **cause** cardiovascular disease?”*

Machine Learning Methods

- Optimizing **predictions**
- **Flexible model** assumptions
- Discover unknown patterns and **complex interactions**
- **Non-linear** relationships
- **High-dimensional** datasets ($p > N$)
- Complex models: “**black box**” by nature

*“**Who** will develop cardiovascular disease?”*

! Important

Machine learning can also be used for causal inference, but this is outside the scope of this lecture

Why ML in Healthcare Now?

The Data Revolution

Exponential Growth:

- Electronic health records everywhere
- Genomics becoming routine
- Wearable devices widespread
- High-resolution medical imaging

We're drowning in data but starving for insights

Complex Medical Challenges

New Complexities:

- Multiple chronic conditions per patient
- Rare diseases being recorded
- High dimensional data
- Dynamic health trajectories

Traditional methods hit their limits

Personalized Medicine Era

Individual Focus:

- Risk assessment individualized
- Dosing optimized per person
- Genetic guided therapies
- Tailored prevention strategies

One-size-fits-all is obsolete

Technology Enablers

Infrastructure Ready:

- Cloud computing democratized
- GPU processing power accessible
- Open-source tools proliferated
- Storage costs plummeted

The computational barrier dissolved

Emphasize that this isn't just about having better computers—it's about the convergence of multiple trends creating a unique opportunity for healthcare improvement.

Machine Learning: The Perfect Bridge

Handles High-Dimensional Data *Thousands of variables? No problem.*

Discovers Hidden Patterns *Finds relationships humans miss.*

Automates Feature Learning *Less manual engineering needed.*

Scales with Data Growth *More data = better performance.*

i Note

Before: We had either data OR computing power OR methods—never all three.

Now: Data explosion + cloud computing + ML algorithms = **Healthcare transformation**

This slide serves as a powerful transition from “why now” to “what ML can do.” The bridge metaphor visually reinforces that ML connects our data-rich reality with actionable clinical solutions.

Core Concepts of Machine Learning

Machine Learning Workflow for Epidemiological Research



! Key Insight

When using machine learning it is not just algorithms, it's a **systematic process**. Each step is crucial for success.

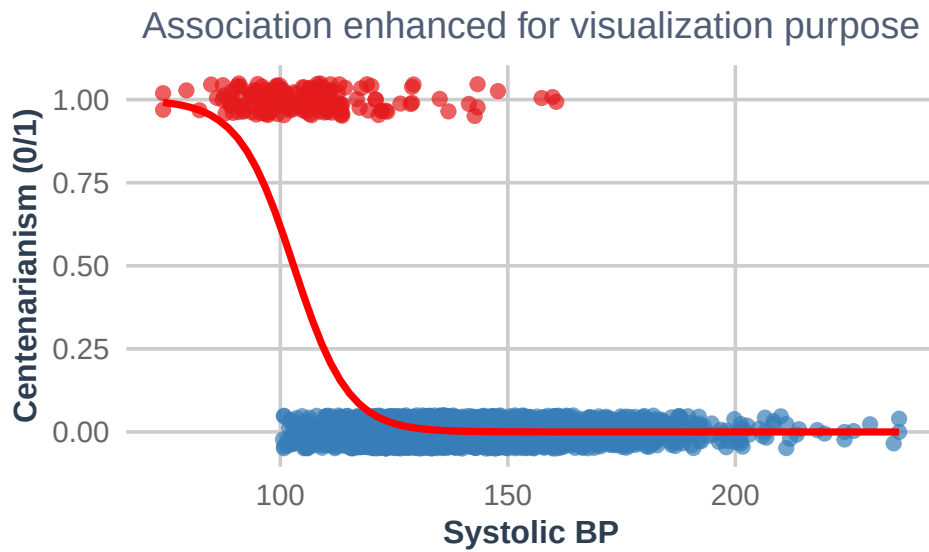
Emphasize that the overall process is similar to traditional epidemiological studies. We will dive deeper into each step in the following slides.

Supervised vs. Unsupervised Learning

Supervised Learning

“Learning with a teacher”

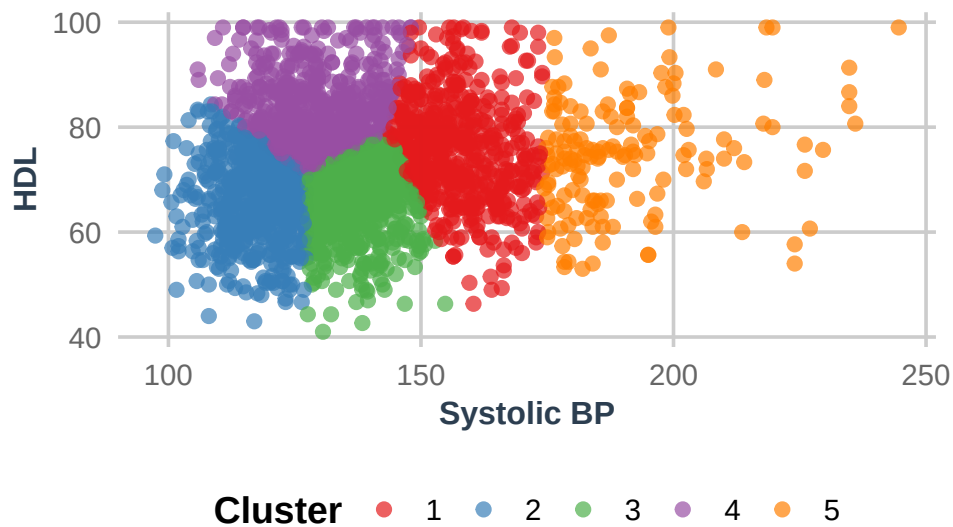
- Known outcomes (labels)
- Learn input → output mapping
- Like medical diagnosis training
- **Examples:** Predicting disease, mortality risk



Unsupervised Learning

“Finding hidden patterns”

- No known outcomes
- Discover structure in data
- Like organizing library without categories
- **Examples:** Patient clustering, pattern discovery



i Note

In this lecture we will focus on supervised learning models.

Supervised learning is like medical school - students learn from cases with known diagnoses. Unsupervised is like research - discovering patterns without knowing what to look for.

Logistic Regression

Mathematical Foundation

Maximizes the likelihood function:

$$\text{Likelihood}(y, \hat{y}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Key Characteristics

- **Linear assumption:** Assumes linear relationship between predictors and log-odds
- **High interpretability:** Coefficients have direct clinical meaning
- **Reseachers involvement:** Manual variable selection and interactions
- **Cannot handle missing data:** Needs complete cases or imputation
- **Limited to $p < N$ scenarios:** Not suitable when predictors exceed observations

Table 1: Logistic Regression Results: Predictors of Centenarianism

Predictor	Odds Ratio	95% CI Lower	95% CI Upper	P-value
Intercept	0.629	0.283	1.395	0.254
Systolic Blood Pressure	1.018	1.013	1.023	< 0.001
med_smoke_statussex-smoker	1.101	0.884	1.371	0.391
med_smoke_status1-10	0.931	0.752	1.153	0.512
med_smoke_status11-20	2.213	1.703	2.876	< 0.001
med_smoke_status20+	2.071	1.627	2.635	< 0.001
Myocardial Infarction	0.905	0.683	1.199	0.487
Diabetes Mellitus	1.110	0.825	1.494	0.491
Body Mass Index	1.015	0.990	1.041	0.235

Loss Function: “The Cost of Being Wrong”

Model training = Optimization. In classical stats we *maximize* likelihood. In ML we typically *minimize* a **loss**.

What is a Loss Function?

Definition: *Quantifies how much we “pay” for incorrect predictions*

- Likelihood rewards models that assign **high probability to what actually happened**
- Negative log converts that into a **penalty**: smaller is better
- Logistic regression training = minimizing **log-loss** on training data
- **Guides model optimization**: Algorithm minimizes this cost
- **Different costs for different errors**: False positives vs. false negatives
- **Medical context**: Missing cancer (high cost) vs. false alarm (lower cost)

Logistic Regression Loss Function

For binary classification (like Centenarianism prediction):

$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Where: - y_i = actual outcome (0 or 1) - p_i = predicted probability - n = number of observations

Intuition: - When $y_i = 1$ and p_i is small $\rightarrow$ Large penalty - When $y_i = 0$ and p_i is large $\rightarrow$ Large penalty - Perfect predictions $\rightarrow$ Zero loss

Key Insight

The loss function directly shapes what the model learns. Choose wisely based on your clinical priorities!

Emphasize that the loss function is like a “feedback signal” that tells the algorithm what mistakes are most costly. In epidemiology, we often care more about false negatives (missing cases) than false positives.

LASSO Regression (Least Absolute Shrinkage and Selection Operator)

Mathematical Foundation

LASSO adds a regularization term to penalize coefficient magnitude:

$$\text{Cost}(y, \hat{y}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \lambda \sum_{j=1}^n |\beta_j|$$

Key Characteristics

- **Automatic feature selection:** Shrinks unimportant coefficients to zero
- **Regularization:** hyperparameter controls shrinkage amount
- **Handles high-dimensional data:** Can work when $p > N$
- **Cross-validation tuning:** Uses CV to select optimal
- **Maintains interpretability:** Similar to logistic regression

Advantages Over Standard Logistic Regression

- Prevents overfitting through regularization
- Automatically selects relevant variables
- Handles multicollinearity better
- Works with high-dimensional datasets

Mathematical Penalty

$$\text{Total Cost} = \text{Loss Function} + \lambda \times \text{Penalty}$$

- λ (lambda): Regularization strength
- Higher λ = More aggressive pruning
- Tuned using cross-validation

Use the gardening analogy - just as pruning makes trees healthier and more robust, regularization makes models more generalizable by removing unnecessary complexity.

Regularization

The Regularization Concept

Core Idea: *Prevent overly complex models*

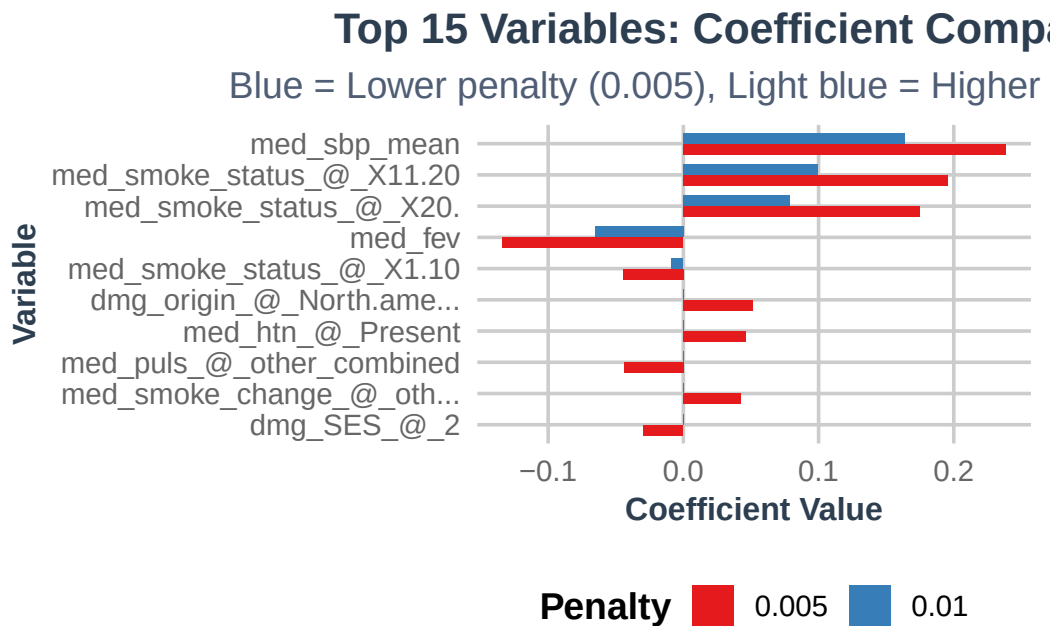
- **Occam's Razor:** Simpler explanations are usually better
- **Biological basis:** Parsimonious models often generalize better
- **Prevents overfitting:** Stops model from memorizing noise

Types of Regularization

LASSO (L1): - Shrinks coefficients toward zero - **Variable selection:** Can set coefficients exactly to zero - Creates sparse models

Ridge (L2): - Shrinks all coefficients toward zero - Keeps all variables but reduces their impact - Good when all predictors are somewhat relevant

Elastic Net: - Combines LASSO and Ridge - Balances variable selection with coefficient shrinkage



The Bias-Variance Trade-off

Key Concepts

Bias: measures how far off, on average, a model's predictions are from the ground truth values.

Variance: measures how much a model's predictions change with different training datasets

Overfitting occurs when variance is high

Underfitting when bias is high **Sweet spot:** Balanced complexity for your data



Bias = systematic error | Variance = prediction inconsistency

The Overfitting Problem (Student Learning Analogy)

Definition: *Model learns noise instead of signal*

Memorizer (Overfitted):

- Knows textbook word-for-word
- Can't answer new questions
- Fails when context changes
- Excellent on practice tests - Poor on real exam

Understander (Well-fitted):

- Grasps underlying concepts
- Applies knowledge to new situations
- Adapts to different contexts
- Good on both practice and real tests

The student analogy is powerful because everyone can relate to the difference between memorizing and understanding. Emphasize that overfitting is the #1 reason ML models fail in practice.

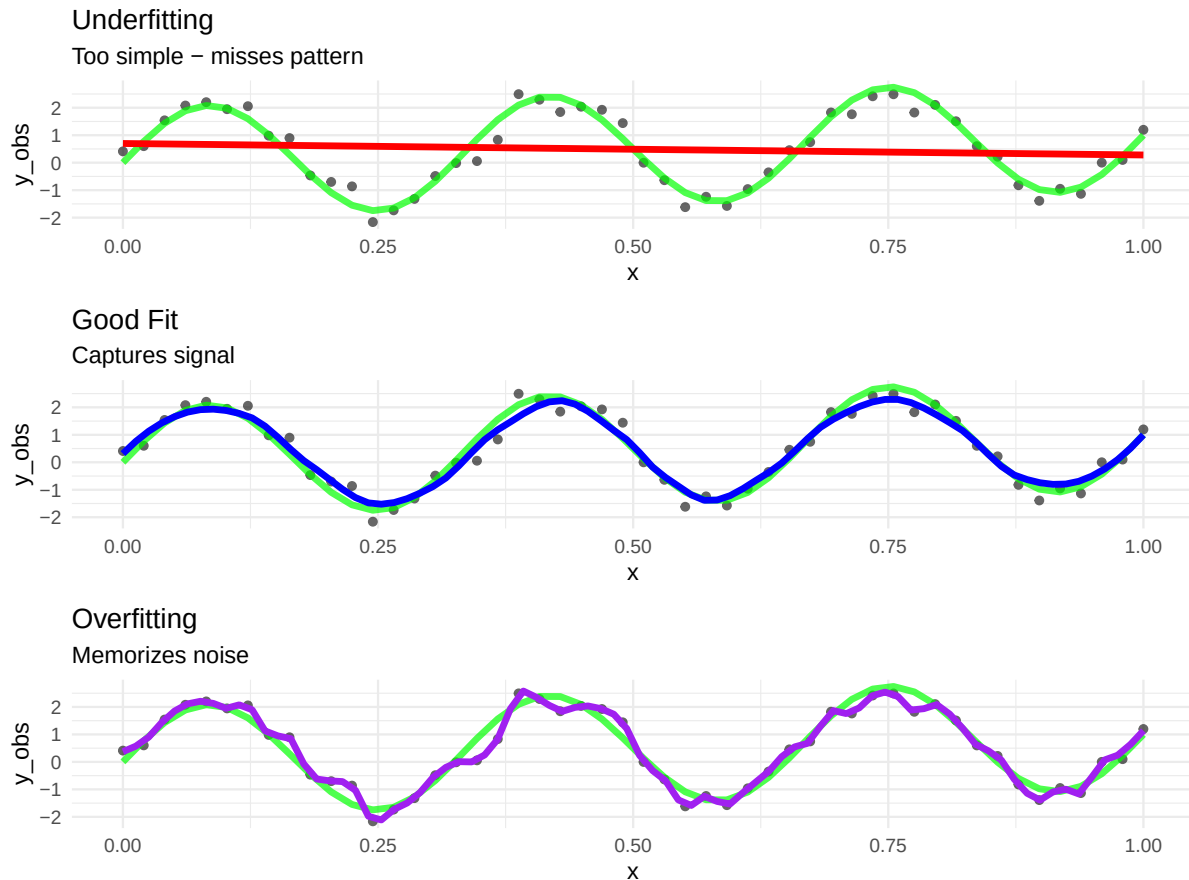
Overfitting: “Memorizing vs. Understanding”

Warning Signs

1. **Perfect training scores:** Usually too good to be true
2. **Large gap:** Training accuracy » Test accuracy
3. **Complex model:** Too many parameters for data size
4. **Unstable predictions:** Small data changes → big prediction changes

Prevention Strategies

- **Validation:** Hold out test data
- **Cross-validation:** Multiple honest estimates
- **Regularization:** Penalize complexity
- **Early stopping:** Stop before overfitting begins



⚠ Remember: Overfitting is ML's Greatest Enemy

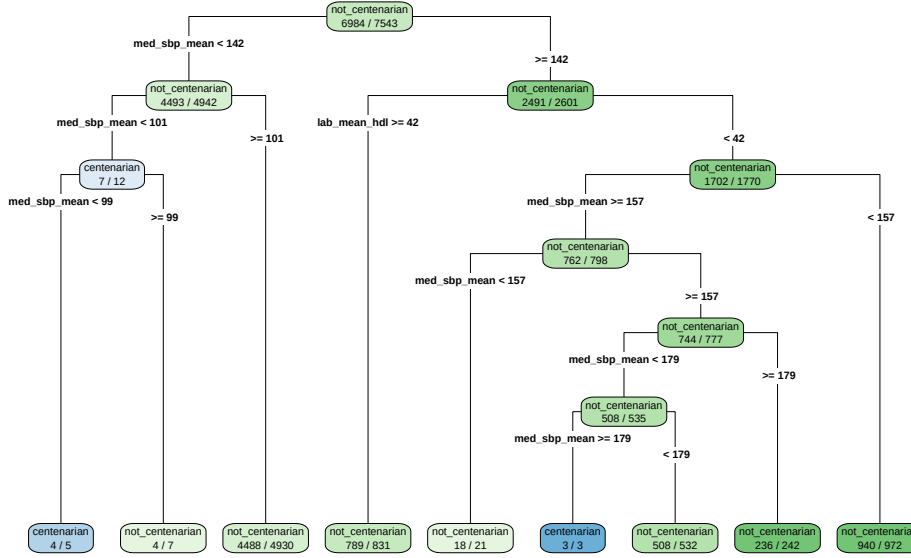
Always validate on truly unseen data. If it seems too good to be true, it probably is!

Decision Trees

Key Characteristics

- **Non-parametric architecture:** No assumptions about data distribution or linear relationships
- **Recursive splitting:** Algorithm selects the variable and split point that maximizes information gain
- **Stopping criteria:** Controlled by minimum samples per node, maximum depth, and complexity parameters (hyperparameters)
- **Handles mixed data types:** Works with both continuous and categorical predictors natively

- **Automatic interaction detection:** Naturally captures variable interactions through hierarchical splits
- **Interpretable:** Easy to understand and explain to clinicians
- **Prone to overfitting:** Thus, rarely used alone for prediction



Decision Trees - under the hood

Mathematical Foundation

Decision trees split data using decision criteria, similar to clinical flowcharts, to create homogeneous (pure) groups with each split. Common metrics include Gini impurity, entropy and information gain.

$$\text{Total Gini} = \sum_{j=1}^L \frac{N_j}{N} \times \left(1 - \sum_{i=1}^c p_{ij}^2 \right)$$

Where:

L = number of leaf nodes in the tree; N_j = number of samples in leaf node j ; N = total number of samples in the dataset; $1 - \sum_{i=1}^c p_{ij}^2$ = Gini impurity of leaf node j ; c = the number of classes; p_{ij} = proportion of class i in leaf node j

Mathematical Intuition

- **Gini = 0**: Perfect classification (all samples same class)
- **Gini = 0.5**: Maximum impurity for binary classification (50-50 split)
- **Perfect tree**: Total Gini = 0 (all leaves are pure)
- **No splits**: Total Gini = original root Gini
- **Larger leaves** contribute more to the total impurity due to weighting by $\frac{N_j}{N}$

eXtreme Gradient Boosting (XGBoost)

Algorithm Overview

XGBoost employs **sequential ensemble learning** where each tree corrects errors from previous learners, building weak decision trees that collectively form a powerful predictor.

Key Innovation: Unlike simple decision trees that make single predictions, **gradient boosting** sequentially trains multiple trees where each corrects errors from previous ones, while XGBoost enhances this with **similarity scores** and built-in **regularization**.

Key Characteristics

- **Ensemble method**: Combines multiple weak learners sequentially
- **Gradient boosting**: Each tree learns from previous trees' errors
- **Built-in regularization**: (λ) prevents overfitting by shrinking similarity scores
- **Automatic pruning**: (γ) removes branches that don't improve gain sufficiently
- **Built-in feature selection**: Automatically prioritizes important variables
- **Handles missing data**: Native missing value support without imputation
- **Complex relationships**: Captures non-linear patterns and interactions

eXtreme Gradient Boosting (XGBoost)

Advantages in Epidemiology

- **Excellent predictive performance** with complex health datasets
- **Handles large, high-dimensional data** common in electronic health records
- **Automatic interaction detection** between risk factors
- **Robust to outliers and missing data** (crucial for real-world medical data)

Limitations

- **Black box nature:** Requires post-hoc explanation methods (SHAP)
- **Computational intensity:** More resources than simpler models
- **Hyperparameter sensitivity:** Multiple parameters (, , , max_depth) require tuning
- **Overfitting risk:** Despite regularization, still needs careful cross-validation

XGBoost - under the hood

How XGBoost Works

Step 1: Start with initial prediction (typically 0.5)

Step 2: Calculate **pseudo-residuals** (errors from previous predictions)

Step 3: Build trees to predict residuals using **similarity scores**: $\text{Similarity} = \frac{(\sum \text{residuals})^2}{\text{number of residuals} + \lambda}$

Step 4: Use **gain** to find optimal splits: $\text{Gain} = \text{Similarity}_{\text{left}} + \text{Similarity}_{\text{right}} - \text{Similarity}_{\text{root}}$

Step 5: **Prune** using gamma (complexity parameter)

Step 6: Scale by learning rate () and repeat

Key Mathematical Concepts

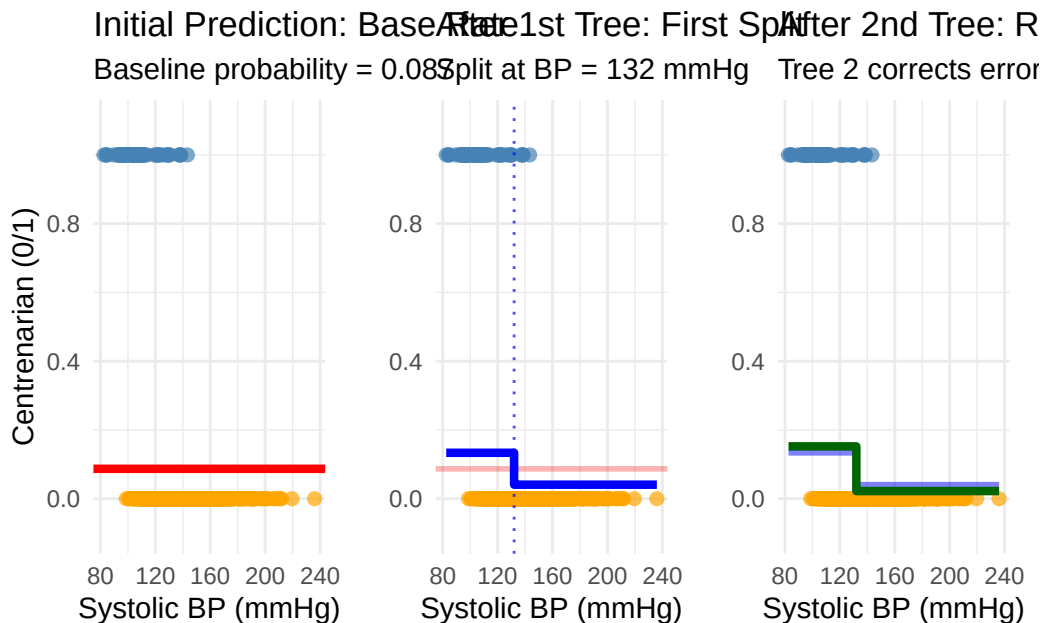
Similarity Scores: When residuals are similar, they don't cancel out → high similarity score. When residuals are different, they cancel out → low similarity score.

Learning Rate (): Takes “small steps in the right direction” - empirical evidence shows many small steps outperform few large steps.

Regularization Effects:

- **Lambda () > 0:** Shrinks similarity scores → easier pruning → prevents overfitting
- **Gamma ():** Sets minimum gain threshold for splits → controls tree complexity

XGBoost - under the hood



Hyperparameters: “Adjusting the Recipe”

What are Hyperparameters?

Definition: *Settings chosen before training begins*

- **Not learned from data:** We set them manually
- **Control learning process:** How the algorithm behaves
- **Recipe analogy:** Like oven temperature and cooking time

Seen so far - **Logistic regression:** regularization strength (λ) or $C = 1/\lambda$; solver/optimizer

- **LASSO:** the **penalty** that shrinks/zeros coefficients (sparsity control)

- **XGBoost (gradient-boosted trees):** learning rate (*eta*), tree depth, number of boosting rounds, subsampling, etc.

How to Determine Hyperparameters

- **Define evaluation plan first** (*prevent leakage*)

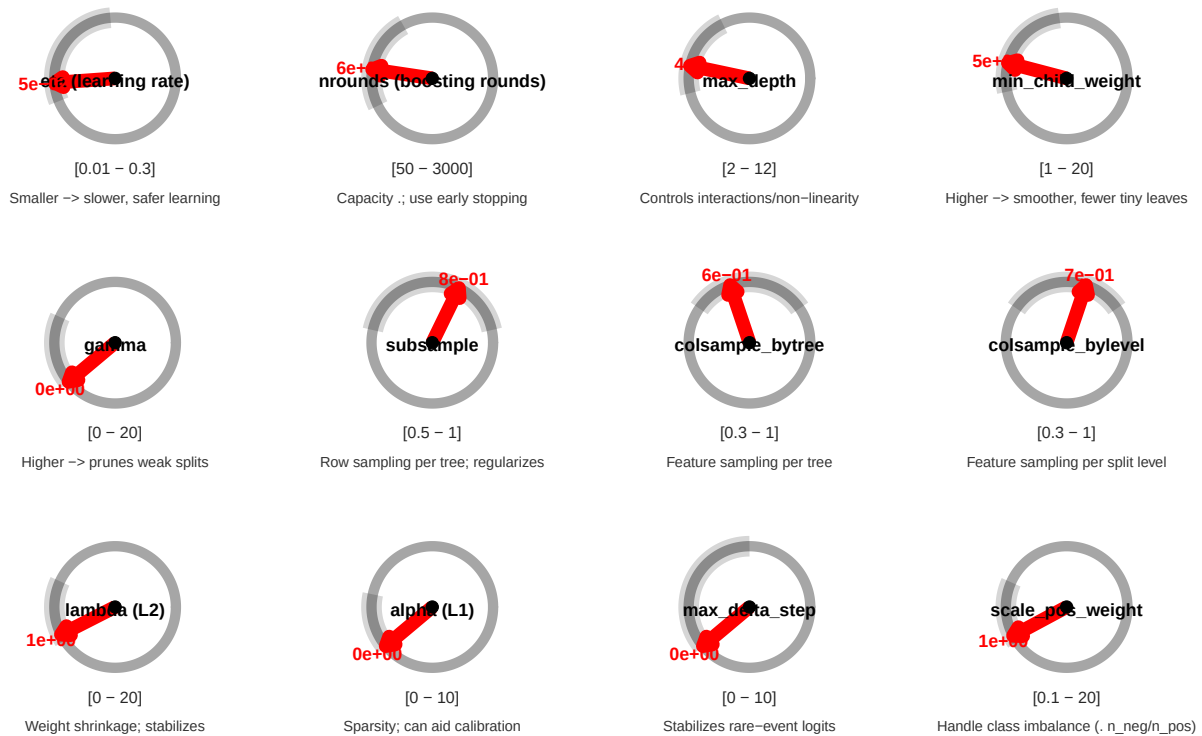
- Prefer **temporal/site splits** over random splits when data are time/site-structured
- Use **nested cross-validation** for honest performance when tuning is heavy
- **Search strategies**
 - **Grid search:** small spaces, few params
 - **Random search:** better coverage when spaces are wide
 - **Bayesian optimization:** sample efficiently near promising regions
 - **Early stopping:** stop boosting when validation loss stops improving
- **Choose using fit-for-use metrics**
 - Discrimination (*AUC*) **and calibration** (reliability curves/Brier)
 - **Decision Curve Analysis** to align thresholds with clinical utility
 - Check **fairness** & subgroup stability

Emphasize that hyperparameter tuning is like being a chef - you need to adjust the recipe based on ingredients (data) and desired outcome (prediction task). There's no universal "best" setting.

Tuning Process

XGBoost Hyperparameter Dials – Chosen Values, Ranges, and Notes

Shaded arc = commonly effective region; needle = your current choice



Model Comparison Summary

Table 1. Comparison of data analysis algorithms

Aspect	Logistic Regression (LR)	LASSO	XGBoost
Variable selection	Manual; relies on researcher's expertise.	Automatically shrinks some coefficients to zero.	Selects variables by contribution to the loss function during training.

Aspect	Logistic Regression (LR)	LASSO	XGBoost
Interactions	Must be prespecified by the researcher.	Same as LR.	Captured automatically within tree structures.
Linearity	Assumes linearity (extendable via polynomials/splines).	Similar to LR; allows some non-linearity.	Handles non-linearity well; invariant to monotonic transformations.
Missing data	Requires imputation or removal before modeling.	Same as LR.	Handles missing data natively via optimal split direction.
Variable transformation	Continuous variables may need scaling; categorical require one-hot encoding.	Same as LR.	Only categorical variables require one-hot encoding.
Overfitting	Low risk but no built-in control.	Reduces overfitting via coefficient shrinkage.	Controlled by internal regularization and sequential weak learners.
Interpretability	High interpretability.	High interpretability.	Low; requires explainable AI tools (e.g., SHAP).
Large number of variables	Inefficient when $p > N$.	Handles high-dimensional data ($p > N$).	Handles high-dimensional and complex data well.
Large number of samples	Computationally efficient.	Moderate computational cost.	Computationally intensive.

Train/Test Split and Cross-Validation for Hyperparameter Tuning

Train/Test Split

- Divide data into training and testing sets to evaluate model performance on unseen data.

- Common splits: 70/30, 75/25, 80/20.
- Can be random or based on time/site to prevent leakage.

K-Fold Cross-Validation

- Further splits training data into k-folds (e.g., 5 or 10) to tune hyperparameters and assess model stability.
- Each fold used once as validation while the rest form the sub-training set.
- Average performance across folds estimates generalization error.
- Choose metrics aligned with clinical goals (e.g., Sensitivity, AUC, calibration).

Hyperparameter Grid Search

- Define a grid of hyperparameter combinations to evaluate.
- Use cross-validation to assess each combination.
- Select combination with best average performance on validation folds.

